

Causally Ordered Security-State Theory (COSS)

Nicola Gallo

27 September 2026

Abstract

Security models are usually presented as separate mechanisms: capability and possession systems constrain authority, mandatory access-control and information-flow systems constrain propagation of information, relational authorization constrains which classes of executors may continue an operation, and provenance systems constrain which artifacts or production histories may participate in an execution. Their state spaces and enforcement rules differ, and classical results can therefore appear to place these models in conflict.

This paper proposes Causally Ordered Security-State Theory (COSS), a general continuity construction in which security systems contribute coordinates to a product security state carried by a causally ordered execution. PIC places authority in the event space $O \times R \times L$ of privileges exercised within causal lineages. COSS keeps the lineage axis and replaces $O \times R$ by the product $\mathcal{B}$ of the behavior universes of the carried security systems: every carried security state, not only authority, is attached to and attenuated along causal history. Each coordinate retains its own refinement relation; ordinary continuation is valid only when the next execution occurrence is causally related to its predecessor and no coordinate becomes more permissive according to its own order. For lineages consisting of ordinary continuations, the resulting product order yields an origin-bounded continuity theorem by componentwise transitivity; explicitly authorized relaxation events restart that bound at an auditable causal point.

The construction generalizes Provenance Identity Continuity (PIC): the original PIC model is recovered when the product has a single authority coordinate ordered by set inclusion, while the relational extension of PIC is recovered as a two-coordinate product of authority and relational admissibility. A projection/factorization theorem then shows more generally that a policy which observes only a projection of the product state cannot enforce a property that distinguishes states inside the same projection fiber. This gives a precise way to reinterpret the historical sequence from Bell–LaPadula and Boebert through Miller, Yee, and Shapiro’s capability analysis and Miller and Shapiro’s access abstractions: the results need not be read as choosing one security model over another, but as reasoning over different observable security distinctions and the mechanisms that make those distinctions available to enforcement.

1 Introduction

Security mechanisms answer different questions. A capability can answer whether an executor possesses authority over an object. An information-flow policy can answer whether information at one security class may flow to another. A relational authorization rule can restrict which kind of executor may become the next participant. A provenance rule can restrict which artifact, builder, or production history may be used. These are not merely different encodings of one scalar security property. They are different security state spaces.

The central proposal of this paper is to stop requiring one such state space to subsume the others. Instead, let each security system contribute one coordinate to the state of a concrete execution occurrence, and let causal continuity transport the product state.

Informally, if $S_1, \dots, S_m$ are security-state systems, an occurrence at step i carries

$$\Sigma_i = (\sigma_i^{(1)}, \dots, \sigma_i^{(m)}) \in \prod_{j=1}^m X_j, \quad (1)$$

where X_j is the state space of S_j . Every system supplies its own “no-more-permissive-than” relation $\preceq_j$. Ordinary causal continuation requires

$$\text{PoR}(s_i, s_{i+1}) \quad \wedge \quad \forall j, \sigma_{i+1}^{(j)} \preceq_j \sigma_i^{(j)}. \quad (2)$$

The first conjunct binds the states to one causal execution; the second prevents any security coordinate from expanding silently while that execution continues. At the event level, this is the lift from PIC’s $O \times R \times L$ to $\mathcal{B} \times L$: the causal axis is preserved, while the behavior judged at an occurrence may now carry facets for every security system in the product (Section 3.3).

This generalizes the continuity pattern already present in PIC. The first PIC model carries an authority context $C_i \subseteq O \times R$ and requires $C_{i+1} \subseteq C_i$ along a PoR-linked execution [10]. Its relational extension already replaces the scalar authority coordinate by the product state $(C_i, [\rho_i]_{\text{sem}})$ and orders that product componentwise by set inclusion and logical implication [11]. COSS takes the next step: the product itself is the general object, while authority is one possible coordinate.

The contribution is not the observation that security systems can be combined. The contribution is a minimal

mathematical condition under which their states can be *continued together without collapsing their distinct semantics*: causal linkage plus componentwise non-expansion.

Contributions. This draft develops seven results.

1. An explicit lift of PIC’s event space $O \times R \times L$ to a general security-event space $\mathcal{B} \times L$.
2. A general ordered security-state coordinate and its product construction, with a joint product-admission semantics.
3. A Product Continuity theorem showing that every ordinary descendant is origin-bounded in every coordinate and jointly in admitted behavior.
4. An application of the standard principal down-set embedding, showing that arbitrary partial-order refinement can be represented faithfully as inclusion of principal lower sets.
5. A projection/factorization theorem and lineage-forgetting corollary showing that a policy blind to a coordinate cannot enforce distinctions erased by that projection.
6. A translated-continuity theorem for heterogeneous security-state spaces.
7. A segment-continuity theorem for lineages containing explicit authorized relaxations, together with derivations of PIC, relational PIC, capability authority, and an information-flow coordinate as instances of the same causal transport construction.

2 Historical Motivation: Different Security Coordinates

Bell and LaPadula developed a mathematical model of secure computer states and rules intended to preserve security under state transitions [1, 2]. Denning subsequently formulated secure information flow using a lattice of security classes, giving a general mathematical view of systems that restrict information flow [3].

Capability systems start from a different primitive. Authority is represented by protected designators that permit operations on objects. In 1984, Boebert showed that an *unmodified* capability machine, in the absence of additional mechanisms, cannot enforce the Bell–LaPadula *-property in his model [4]. His concrete attack lets a low-clearance program place a read/write capability for a low object into that low object; a later high-clearance program may read the low object, obtain the capability, and then combine read access to high data with write access to the low object. Boebert’s concluding explanation is that, in the pure capability machine he defines, the right to exercise access carries the ability to propagate that access.

Miller, Yee, and Shapiro revisited this construction directly in *Capability Myths Demolished* [5]. They agree that the particular construction described by Boebert fails to enforce the *-property, but argue that the attack assumes capabilities can be transmitted wherever data can be transmitted. In partitioned or type-enforced capability systems, capability transfer and data transfer are distinguishable operations. Their two-level construction independently controls those operations and is presented as a counterexample to the broader claim that capability systems as a class cannot enforce the *-property. The same paper also stresses that a static access matrix is not a complete description of a security mechanism: the rules by which access relationships evolve over time materially affect the security properties of the resulting system.

Miller and Shapiro develop a complementary argument in *Paradigm Regained* [6]. They distinguish permission from authority and explicitly make program behavior on permitted causal pathways part of authority analysis. Their object-capability model carries overt causation by messages over permitted paths. Their examples use Redell’s Caretaker pattern for revocation, factories for overt confinement, and a data diode for the *-properties. They also show how multiple independently motivated abstractions can coexist over the same graph under “mutually suspicious composition.”

These papers therefore already contain dynamic, behavioral, and causal reasoning; COSS does not claim otherwise. The distinction made here is representational. In those analyses, causal pathways and abstraction behavior are derived from the reference graph and program semantics. COSS instead makes causal lineage an explicit axis of the security event and transports an ordered security state along that axis. Nor does COSS assume that a capability implementation can contribute only an authority coordinate: a capability design may itself mediate information flow, transfer type, request context, or another security distinction. The general claim is narrower: if an enforcement decision is invariant under a distinction that changes the target security verdict, that decision cannot enforce the target property. A typed transfer rule or access abstraction may expose exactly the distinction that a coarser model omitted.

This paper does *not* claim that Boebert, Bell–LaPadula, Denning, Miller, or Shapiro formulated their work in this product language. The product view is a new interpretation intended to explain why apparently conflicting results can all be correct under different observable state spaces.

3 Preliminaries

3.1 Execution occurrences and causal continuity

Let S be a set of execution occurrences. An occurrence is a concrete step of an execution, not merely the identity

of the executor that realizes it. Let

$$\text{PoR} \subseteq S \times S \quad (3)$$

be the immediate causal-continuation relation. We write $\text{PoR}(s_i, s_{i+1})$ when s_{i+1} is in fact a valid causal continuation of s_i within the modeled execution.

A concrete enforcement mechanism may establish PoR from signatures, attestations, observed handoffs, trusted-runtime evidence, or another witness. Verification and the relation it establishes are distinct: the abstract theory assumes sound verification, meaning that every admitted transition for which the mechanism establishes $\text{PoR}(s_i, s_{i+1})$ satisfies the modeled causal relation.

A finite execution lineage is

$$\pi = \langle s_0, s_1, \dots, s_n \rangle \quad (4)$$

with $\text{PoR}(s_i, s_{i+1})$ at every adjacent step. Let L denote the set of such finite causal histories. The concrete construction of PoR is orthogonal to the abstract results below; as in PIC, the theorems are conditional on sound causal verification [10, 11].

3.2 Occurrence coordinates versus security-state coordinates

An occurrence may additionally be observed through coordinates that are not themselves attenuation coordinates, for example

$$\omega_i = (\ell_i, \iota_i, g_i, \Sigma_i), \quad (5)$$

where ℓ_i is causal position or lineage information, ι_i is an executor or identity observation, g_i is a geographic or network location, and Σ_i is the security state carried by the occurrence.

Lineage grows by causal extension rather than attenuation, while the current identity or location may vary from step to step. If geography is a security restriction, the monotone object is instead an admissible set or predicate, e.g.

$$g_i \in G_i^{\text{allow}}, \quad G_{i+1}^{\text{allow}} \subseteq G_i^{\text{allow}}. \quad (6)$$

This distinction prevents the causal coordinates of an occurrence from being confused with the policy state that constrains them.

3.3 From $O \times R \times L$ to $\mathcal{B} \times L$

In PIC an authority event is an occurrence of a privilege within a causal lineage,

$$e = (o, r, \ell) \in O \times R \times L, \quad (7)$$

and the state carried by an occurrence is an authority context $C \subseteq O \times R$ [10]. The privilege coordinates $O \times R$ say *what* is exercised; the lineage coordinate L says *in which causal history* it is exercised. COSS keeps the lineage axis unchanged and replaces the single privilege

universe by the product of the behavior universes of all carried security systems.

Anticipating Definition 2, let each carried security system contribute a behavior universe $\mathcal{B}_j$.

Definition 1 (Security event). *Let $\mathcal{B} = \prod_{j \in J} \mathcal{B}_j$. A security event is*

$$e = (b, \ell) \in \mathcal{B} \times L, \quad (8)$$

where ℓ is the causal history of the occurrence and $b = (b^{(j)})_{j \in J}$ collects, for each coordinate, the facet of the occurrence that the coordinate judges: the privilege it exercises, the flow it performs, the executor transition it realizes, or the location at which it occurs.

For the single authority coordinate, $\mathcal{B}_{\text{auth}} = O \times R$ and $\mathcal{B} \times L = O \times R \times L$: PIC's event space is the one-coordinate instance. The same substitution acts on states, where $\mathcal{P}(O \times R)$ becomes $X = \prod_j X_j$. The construction of PIC is otherwise transported unchanged: states are carried along PoR-linked occurrences, and ordinary continuation may only refine them.

Remark 1. *If a coordinate imposes nothing on a given occurrence, its facet may be a distinguished element $\perp_j \in \mathcal{B}_j$ admitted by every state of that coordinate.*

4 Ordered Security-State Systems

Definition 2 (Security-state coordinate). *A security-state coordinate is a tuple*

$$\mathfrak{S}_j = (X_j, \preceq_j, \mathcal{B}_j, \llbracket \cdot \rrbracket_j), \quad (9)$$

where:

- X_j is a set of security states;
- $\preceq_j$ is a partial order interpreted as “no more permissive than”;
- $\mathcal{B}_j$ is a universe of behaviors, transitions, principals, flows, or other semantically admitted possibilities; and
- $\llbracket x \rrbracket_j \subseteq \mathcal{B}_j$ is the semantic extension admitted by state $x \in X_j$.

The coordinate is semantically monotone when

$$x' \preceq_j x \implies \llbracket x' \rrbracket_j \subseteq \llbracket x \rrbracket_j. \quad (10)$$

The definition deliberately does not require every security system to be born as a powerset. A lattice label, a logical predicate, a capability set, a provenance constraint, or a history-dependent state can all be used if they admit a sound refinement order.

Remark 2 (Coordinates are semantic dimensions). *A coordinate is not required to correspond one-to-one with an implementation mechanism. One mechanism may realize*

several security coordinates, as when a type-enforced capability runtime distinguishes authority from data transfer and capability transfer; conversely, several mechanisms may jointly realize one coordinate. COSS reasons over the security distinctions exposed to the admission decision, not over product names or implementation categories.

Remark 3 (Refinement is not a complete update semantics). The order $\preceq_j$ is the condition needed for the continuity theorem, not a claim that every refinement step is reachable or permitted by the concrete security system. A coordinate may impose a stricter local transition relation $U_j \subseteq X_j \times X_j$ satisfying

$$U_j(x, x') \implies x' \preceq_j x. \quad (11)$$

All ordinary-continuation results below remain valid when transitions must satisfy both U_j and the refinement order. This leaves room for the richer update rules that distinguish dynamic security mechanisms even when they share the same static state representation.

Definition 3 (Product security state). For a finite family J of security-state coordinates, define

$$X = \prod_{j \in J} X_j, \quad \Sigma = (\sigma^{(j)})_{j \in J}. \quad (12)$$

The product refinement order is

$$\Sigma' \preceq \Sigma \iff \forall j \in J, \sigma'^{(j)} \preceq_j \sigma^{(j)}. \quad (13)$$

For a concrete execution, let

$$\text{st} : S \rightarrow X \quad (14)$$

assign exactly one product security state to each execution occurrence, and write $\Sigma_i := \text{st}(s_i)$. If $\ell \in L$ is a finite causal history with endpoint s_n , define

$$\Sigma(\ell) := \text{st}(s_n). \quad (15)$$

Thus the state used by an event is a well-defined function of the endpoint occurrence of its represented history.

Proposition 1 (Product order). If every $(X_j, \preceq_j)$ is a partial order, then $(X, \preceq)$ defined by (13) is a partial order.

Proof. Reflexivity, antisymmetry, and transitivity hold componentwise.

Remark 4 (Coordinates need not be independent). The Cartesian product is an ambient state space, not an assumption that every combination of coordinate states is meaningful. If cross-coordinate constraints permit only states in $Y \subseteq X$, the induced order on Y is still a partial order and Product Continuity applies unchanged to lineages whose states remain in Y . Likewise, if only certain combinations of behavior facets are meaningful, one may restrict the event universe to $\mathcal{B}^* \subseteq \prod_j \mathcal{B}_j$ and interpret product admission on $\mathcal{B}^* \cap \prod_j \llbracket \sigma^{(j)} \rrbracket_j$.

5 Causally Ordered Security-State Continuity

Definition 4 (Valid COSS transition). A transition

$$(s_i, \Sigma_i) \longrightarrow (s_{i+1}, \Sigma_{i+1}) \quad (16)$$

is COSS-valid when

$$\text{PoR}(s_i, s_{i+1}) \quad \wedge \quad \Sigma_{i+1} \preceq \Sigma_i. \quad (17)$$

Equivalently, every security coordinate refines or preserves its predecessor state while the execution advances along a verified causal edge.

Definition 5 (Product security-state continuity). A lineage

$$\Pi = \langle (s_0, \Sigma_0), \dots, (s_n, \Sigma_n) \rangle \quad (18)$$

has product security-state continuity when every adjacent transition is COSS-valid.

Theorem 1 (Product Continuity). If Π has product security-state continuity, then

$$\Sigma_n \preceq \Sigma_{n-1} \preceq \dots \preceq \Sigma_0. \quad (19)$$

Hence, for every coordinate j ,

$$\sigma_n^{(j)} \preceq_j \sigma_0^{(j)}. \quad (20)$$

If the coordinate is semantically monotone, then

$$\llbracket \sigma_n^{(j)} \rrbracket_j \subseteq \llbracket \sigma_0^{(j)} \rrbracket_j. \quad (21)$$

Proof. Every valid transition gives $\Sigma_{i+1} \preceq \Sigma_i$. Transitivity of the product order gives $\Sigma_n \preceq \Sigma_0$. Projection to coordinate j gives (20); semantic inclusion follows from (10).

The theorem states the general continuity invariant for lineages containing only ordinary continuations: causal transport cannot introduce a possibility forbidden by the origin state of any carried security system.

Definition 6 (Product admission). The product state $\Sigma = (\sigma^{(j)})_{j \in J}$ admits $b \in \mathcal{B}$ iff $b^{(j)} \in \llbracket \sigma^{(j)} \rrbracket_j$ for every j . Write

$$\llbracket \Sigma \rrbracket := \prod_{j \in J} \llbracket \sigma^{(j)} \rrbracket_j \subseteq \mathcal{B}. \quad (22)$$

Proposition 2 (Joint semantic monotonicity). If every coordinate is semantically monotone, then $\Sigma' \preceq \Sigma$ implies $\llbracket \Sigma' \rrbracket \subseteq \llbracket \Sigma \rrbracket$. Consequently, under product security-state continuity, $\llbracket \Sigma_n \rrbracket \subseteq \llbracket \Sigma_0 \rrbracket$.

Proof. $\Sigma' \preceq \Sigma$ gives $\sigma'^{(j)} \preceq_j \sigma^{(j)}$ for every j , hence $\llbracket \sigma'^{(j)} \rrbracket_j \subseteq \llbracket \sigma^{(j)} \rrbracket_j$ by semantic monotonicity, and a product of inclusions is an inclusion. The second claim follows from Product Continuity.

A later occurrence therefore admits only behaviors that the origin state admits in every coordinate jointly, not merely coordinate by coordinate.

6 Why Refinement Can Be Read as Set Inclusion

The previous theorem is intentionally stated over partial orders rather than subsets. Nevertheless, the subset intuition can be recovered for every partial order.

Definition 7 (Principal lower-set representation). *For a poset $(X, \preceq)$ define*

$$\downarrow x = \{y \in X \mid y \preceq x\}. \quad (23)$$

Theorem 2 (Principal down-set embedding). *For every poset $(X, \preceq)$, the standard principal down-set map*

$$\eta : X \rightarrow \mathcal{P}(X), \quad \eta(x) = \downarrow x \quad (24)$$

is an order embedding:

$$x' \preceq x \iff \downarrow x' \subseteq \downarrow x. \quad (25)$$

Proof. If $x' \preceq x$ and $y \preceq x'$, transitivity gives $y \preceq x$, so $\downarrow x' \subseteq \downarrow x$. Conversely, $\downarrow x' \subseteq \downarrow x$ implies $x' \in \downarrow x$ because $x' \in \downarrow x'$ by reflexivity. Hence $x' \preceq x$.

This standard principal down-set embedding formalizes the subset intuition behind the present work. A security system need not literally store a set, but its refinement order can be represented faithfully as inclusion among sets of states no more permissive than the current one. This inclusion must not be confused with semantic admission: $\downarrow x \subseteq X$ is a set of *states*, whereas $\llbracket x \rrbracket \subseteq \mathcal{B}$ is a set of *behaviors*. Semantic monotonicity connects refinement to behavior inclusion, but the two constructions are distinct. The familiar PIC rule $C_{i+1} \subseteq C_i$ is therefore one concrete instance of a more general ordered refinement pattern.

7 Projection, Quotients, and Co-ordinate Blindness

The second PIC paper interprets lineage-forgetting as a quotient and shows that lineage-invariant decisions factor through the projection that forgets causal history [11]. The same construction applies to arbitrary products of security-state coordinates.

Let

$$X = \prod_{j \in J} X_j \quad (26)$$

and let $K \subseteq J$. Define the coordinate projection

$$p_K : X \rightarrow X_K := \prod_{j \in K} X_j. \quad (27)$$

Definition 8 (K -invariant decision). *A decision rule $A : X \rightarrow \{0, 1\}$ is K -invariant when*

$$p_K(x) = p_K(y) \implies A(x) = A(y). \quad (28)$$

Theorem 3 (Projection Factorization). *A decision rule $A : X \rightarrow \{0, 1\}$ is K -invariant if and only if there exists a unique rule*

$$\bar{A} : p_K(X) \rightarrow \{0, 1\} \quad (29)$$

such that

$$A = \bar{A} \circ p_K. \quad (30)$$

Proof. If $A = \bar{A} \circ p_K$, equal projections yield equal decisions. Conversely, define $\bar{A}(z) = A(x)$ for any x with $p_K(x) = z$. K -invariance makes the definition independent of the representative. Uniqueness follows because p_K is surjective onto $p_K(X)$.

Corollary 1 (Omitted-coordinate impossibility). *Suppose a target security property $P : X \rightarrow \{0, 1\}$ distinguishes two states $x, y \in X$ for which $p_K(x) = p_K(y)$. No K -invariant decision rule can equal P on all of X .*

Proof. Every K -invariant rule must give the same result on x and y , while P requires different results.

This is the general form of the earlier lineage-forgetting observation: a security decision cannot enforce a distinction that its observation function erases.

Corollary 2 (Behavioral-state blindness). *Let $X = X_{\text{prot}} \times X_{\text{beh}}$ separate a primitive protection-state coordinate from a coordinate describing security-relevant behavior or an access abstraction. If a target property $P : X \rightarrow \{0, 1\}$ satisfies*

$$P(p, h_1) \neq P(p, h_2) \quad (31)$$

for some fixed $p \in X_{\text{prot}}$, then no decision rule that factors through the projection $(p, h) \mapsto p$ can implement P on all of X .

Proof. This is Omitted-coordinate Impossibility with $K = \{\text{prot}\}$.

This corollary gives an order-independent statement of the blind spot discussed by Miller and Shapiro: if two systems have the same protection graph but the behavior of a security-enforcing abstraction changes the authority or flow verdict, an analysis that observes only the protection graph is necessarily too coarse for that distinction [6]. It does not imply that protection-state analysis is unsound when it succeeds; it states only that such a projection cannot reproduce a behavior-sensitive property in general.

Using the endpoint-state assignment of (15), the COSS admission rule for an event (b, ℓ) is

$$A_{\text{COSS}}(b, \ell) = 1 \iff b \in \llbracket \Sigma(\ell) \rrbracket. \quad (32)$$

Let $q_L : \mathcal{B} \times L \rightarrow \mathcal{B}$, $(b, \ell) \mapsto b$, forget the lineage.

Corollary 3 (Lineage-forgetting blindness in COSS). *Let $b \in \llbracket \Sigma(\ell) \rrbracket \setminus \llbracket \Sigma(\ell') \rrbracket$. No decision rule that factors through q_L equals A_{COSS} on both (b, ℓ) and (b, ℓ') .*

Proof. $q_L(b, \ell) = q_L(b, \ell')$, so every rule factoring through q_L gives both events the same verdict, while $A_{\text{COSS}}(b, \ell) = 1$ and $A_{\text{COSS}}(b, \ell') = 0$. This is the same fiber argument as Projection Factorization, now applied to the projection that omits L from the event space.

For the authority coordinate alone this is the lineage-invariance result of PIC [10]. In COSS it holds for every carried security system at once: the same behavior can be admissible in one causal history and inadmissible in another, in authority, in information flow, in relational admissibility or in any other coordinate, and a decision that forgets the history cannot tell the two apart.

7.1 A three-coordinate example

Consider a three-step agent lineage

$$s_0 \rightarrow s_1 \rightarrow s_2 \quad (33)$$

carrying authority, information-flow, and geographic coordinates

$$\Sigma_i = (C_i, F_i, G_i), \quad \Sigma_2 \preceq \Sigma_1 \preceq \Sigma_0. \quad (34)$$

At s_2 , suppose C_2 still permits the operation (send, reportAPI) and G_2 admits the agent's current EU execution region, but F_2 excludes the flow facet f_{ext} representing transmission of the current data to an external sink. Let b be the event facet tuple whose authority component is that send operation, whose flow component is f_{ext} , and whose geographic component is the current EU region. Authority and geography admit their respective facets, but IFC does not, so

$$b \notin \llbracket \Sigma_2 \rrbracket \quad (35)$$

and the product rejects the event. An authority-only decision would admit the event even though the complete product state rejects it: the authority coordinate itself remains valid, while the IFC coordinate does not.

Now consider a different causal history ℓ' carrying the same authority and geographic states but an IFC state F' that admits f_{ext} , for example after a separately authorized declassification. If ℓ denotes the first history ending at s_2 , the same event facet tuple b may satisfy

$$b \in \llbracket \Sigma(\ell') \rrbracket \quad \text{while} \quad b \notin \llbracket \Sigma(\ell) \rrbracket. \quad (36)$$

The example exhibits both claims of the theory: all relevant security coordinates constrain one causal occurrence jointly, and a decision that forgets the lineage cannot distinguish histories whose current admissibility differs.

8 Classical Security Models as Coordinates

8.1 Authority and capabilities

Let O be operations and R resources. Define

$$X_{\text{auth}} = \mathcal{P}(O \times R), \quad C' \preceq_{\text{auth}} C \iff C' \subseteq C. \quad (37)$$

This coordinate says which authority may participate in the current causal execution. It does not identify the executor that happens to possess other ambient authority.

With this coordinate alone, COSS-validity reduces to

$$\text{PoR}(s_i, s_{i+1}) \quad \wedge \quad C_{i+1} \subseteq C_i, \quad (38)$$

which is precisely the abstract continuity rule of PIC [10].

This specialization is compatible with the stronger object-capability results in the literature. *Capability Myths Demolished* identifies absence of ambient authority, inseparability of designation and authority, and access-controlled delegation channels as important properties of object-capability systems; it also emphasizes that authority conveyed in the context of a request helps avoid the classical confused-deputy pattern [5]. COSS does not treat those protections as missing. A capability system may realize several security distinctions at once; the lineage coordinate matters only for a target property whose verdict also varies with causal execution after the capability-level distinctions have been fixed.

8.2 Information-flow admissibility

Let D be a domain of information sources and sinks, security classes, or other flow endpoints. One possible semantic IFC coordinate is the set of currently admitted flows

$$F \subseteq D \times D. \quad (39)$$

Define

$$F' \preceq_{\text{ifc}} F \iff F' \subseteq F. \quad (40)$$

A Bell–LaPadula or lattice policy can induce such an admissible-flow relation; this paper does not replace the internal semantics of those models. It treats their effective admissibility state as an independent coordinate whose ordinary continuation must not become more permissive.

An alternative is to take lattice labels or complete IFC policy states as X_{ifc} and define $\preceq_{\text{ifc}}$ directly from their semantic flow sets. Equation (40) is therefore an example, not a claim that all IFC implementations literally store F .

8.3 Typed data and capability transfer

The distinction used by Miller, Yee, and Shapiro to revisit Boebert can itself be represented as an ordered security coordinate without claiming that their systems used this notation. Let Λ be a set of security domains and let

$$K = \{\text{data}, \text{cap}\}, \quad \mathcal{B}_{\text{tr}} = \Lambda \times \Lambda \times K \quad (41)$$

classify a transfer by source domain, destination domain, and whether the transferred value is ordinary data or a capability. Define

$$X_{\text{tr}} = \mathcal{P}(\mathcal{B}_{\text{tr}}), \quad (42)$$

$$Q' \preceq_{\text{tr}} Q \iff Q' \subseteq Q, \quad \llbracket Q \rrbracket_{\text{tr}} = Q. \quad (43)$$

The coordinate can therefore admit a data transfer between two levels while rejecting a capability transfer over the same pair of levels.

This captures the specific distinction made in *Capability Myths Demolished*: Boebert’s attack requires a high-clearance subject to obtain, by reading a low object, a capability that confers write access to that low object; the later type-enforced construction distinguishes capability movement from data movement and blocks the cross-level capability transfer [5]. The coordinate Q is only an illustrative semantic encoding of that distinction, not a claim that an object-capability implementation literally stores a set of type flows.

8.4 Relational admissibility

The relational extension of PIC carries a predicate ρ_i over successor/predecessor profiles and orders predicates by implication [11]. After quotienting by semantic equivalence, define

$$X_{\text{rel}} = \text{Pred}(\mathcal{T}_J) / \equiv_{\text{sem}}, \quad (44)$$

with

$$[\rho'] \preceq_{\text{rel}} [\rho] \iff \rho' \Rightarrow \rho. \quad (45)$$

Semantically,

$$\rho' \Rightarrow \rho \iff \llbracket \rho' \rrbracket \subseteq \llbracket \rho \rrbracket. \quad (46)$$

Thus the existing PIC product

$$\Gamma_i = (C_i, [\rho_i]_{\text{sem}}) \quad (47)$$

is already a two-coordinate instance of COSS.

8.5 Geographic, provenance, and product constraints

A geographic restriction can be represented by an admissible region set G_i^{allow} ordered by inclusion, while the observed current location g_i is checked for membership. A software-product or provenance coordinate can analogously represent the set of admissible artifact identities, builders, attestations, or derivation histories. These coordinates do not need to share an ontology with authority or IFC; they need only expose a sound refinement order.

This distinction is useful for invocation. A product can participate in many runtime invocations, just as one executor can realize many execution occurrences. Product identity is therefore not itself an invocation. A future DAG extension can treat invocation as a causal join between an authority lineage and a product/provenance lineage while preserving the coordinates that justify each parent.

9 Re-reading the Capability/IFC Tension

Consider the two-coordinate state

$$\Sigma = (C, F) \in X_{\text{auth}} \times X_{\text{ifc}}. \quad (48)$$

A decision that reads only C factors through the projection

$$p_C(C, F) = C. \quad (49)$$

If two states (C, F_1) and (C, F_2) require different information-flow verdicts, any C -invariant mechanism must nevertheless treat them identically. This is not a statement that capability systems are intrinsically incapable of information-flow enforcement. It is a statement about observational insufficiency: the decision cannot enforce a distinction represented only in a coordinate it does not observe.

This gives a compact reinterpretation of the historical sequence without turning it into a contest between security paradigms. Boebert’s oracle does observe clearance and classification when it grants a capability; the failure in his construction occurs in subsequent propagation. Even an oracle that correctly creates capabilities cannot, in the unmodified machine he defines, control their further propagation [4]. The distinction here is therefore between the grant decision and what later executions may acquire and exercise, not between a labeled and an unlabeled grant.

The later capability responses act directly on that propagation assumption. Miller and Shapiro’s data diode transmits data in one direction and capabilities in neither [6]; Miller, Yee, and Shapiro separately show how distinguishing capability transfer from data transfer blocks the cross-level capability movement required by Boebert’s attack, while remaining within their reading of his definition [5]. Under a COSS encoding, an oracle grant that enlarges the carried authority would be an origination or explicitly authorized relaxation, whereas acquisition of a new capability during ordinary continuation cannot silently enlarge the authority coordinate. This is a COSS interpretation of the security distinction, not a claim that Boebert or the later capability papers used this state model.

Miller and Shapiro further show that program behavior and access abstractions can tighten authority beyond what an arrangement-only analysis of the protection graph predicts [6]. In COSS terms, these constructions are examples of mechanisms that make additional distinctions observable or restrict the semantic state on which the decision operates. Once those distinctions are observed, the mechanism no longer belongs to the projection-blind class covered by the impossibility corollary. The blindness claim concerns the information read when an event is admitted; it does not say that the original capability-granting oracle ignored clearance or classification.

The same reasoning generalizes. If

$$\Sigma = (S_1, S_2, \dots, S_m), \quad (50)$$

a policy that factors only through S_1 cannot enforce a property whose verdict varies with S_2 while S_1 is fixed. Security models can therefore be individually valid and still be jointly necessary.

10 PIC as a Derived Theory

The relationship between COSS and PIC is derivational rather than competitive.

PIC, first form. Take a single coordinate

$$X = X_{\text{auth}} = \mathcal{P}(O \times R). \quad (51)$$

Then COSS-validity is exactly PoR plus authority attenuation, and Product Continuity becomes PIC's origin-binding result.

Relational PIC. Take

$$X = X_{\text{auth}} \times X_{\text{rel}}. \quad (52)$$

Then the product order is

$$(C', \rho') \preceq (C, \rho) \iff C' \subseteq C \wedge \rho' \Rightarrow \rho, \quad (53)$$

which is the product attenuation order already used by the relational theory [11].

General form. The natural closure is

$$X = \prod_{j \in J} X_j, \quad (54)$$

where authority, IFC, relational admissibility, provenance, geography, separation-of-duty memory, or another security system may contribute a coordinate. PIC is therefore not renamed or invalidated. It remains the authority-continuity specialization from which the more general product construction was discovered.

11 COSS in the PIC Series

COSS is the third step in a sequence of models that share the same basic shape: authorization is attached to an execution occurrence in causal history, PoR relates adjacent occurrences, a carried state is ordered by refinement, and ordinary continuation may preserve or restrict that state but not silently expand it. At the level of this continuity construction, each later paper contains the earlier state discipline as a specialization; this statement does not imply that the later construction reproduces every implementation theorem or deployment assumption of the earlier papers.

The containment statements are exact at the level claimed here. With $J = \{\text{auth}\}$, the COSS transition rule is PoR plus $C_{i+1} \subseteq C_i$, the validity rule of the first PoC model. With $J = \{\text{auth}, \text{rel}\}$ and relational refinement ordered by implication, the product order is the relational PIC attenuation order $(C', \rho') \preceq (C, \rho)$ iff $C' \subseteq C$ and $\rho' \Rightarrow \rho$ [11].

The separation results share the same factorization principle but should not be identified more strongly than their

domains permit. For the authority-only event space, take $b = (o, r)$ and two histories ℓ, ℓ' such that $(o, r) \in C(\ell)$ and $(o, r) \notin C(\ell')$. The COSS lineage-forgetting corollary then gives the same required separation as the first paper's lineage-invariance result. The relational paper subsequently shows that the earlier lineage-forgetting projection is one quotient instance of a more general factorization construction; its executor-profile quotients and COSS's coordinate projections are applications of the same mathematical principle, not literally the same projection map [11].

The confused-deputy results have the same scope boundary. Hardy's classical example motivates the problem of a deputy exercising authority for the wrong request context [9]. The first PoC paper excludes authority import as valid behavior inside a valid lineage through origin-bounded attenuation. The relational paper separates that model-internal closure from a deployment-level claim about actual request-caused effects: the latter is conditional on Complete Mediation and Causal Attribution [11]. COSS preserves exactly this distinction for its coordinates. Product Continuity constrains states already represented on a lineage; it does not itself determine that an external effect was caused by, or belongs to, that lineage.

12 Heterogeneous Security-State Spaces

PIC already permits policy translation between heterogeneous operation/resource spaces [10]. The same idea extends componentwise.

At hop i , let coordinate j use state space $X_i^{(j)}$ with order $\preceq_i^{(j)}$. Let

$$T_{i \rightarrow i+1}^{(j)} : X_i^{(j)} \rightarrow X_{i+1}^{(j)} \quad (55)$$

be an order-theoretic policy translation. A translated transition satisfies

$$\sigma_{i+1}^{(j)} \preceq_{i+1}^{(j)} T_{i \rightarrow i+1}^{(j)}(\sigma_i^{(j)}). \quad (56)$$

If the translations are monotone and composable, define

$$T_{0 \rightarrow n}^{(j)} = T_{n-1 \rightarrow n}^{(j)} \circ \dots \circ T_{0 \rightarrow 1}^{(j)}. \quad (57)$$

Proposition 3 (Translated continuity). *If every $T_{i \rightarrow i+1}^{(j)}$ is monotone and every transition satisfies (56), then*

$$\sigma_n^{(j)} \preceq_n^{(j)} T_{0 \rightarrow n}^{(j)}(\sigma_0^{(j)}). \quad (58)$$

Proof. By induction on n . For $n = 0$ the claim is reflexivity. Suppose $\sigma_i^{(j)} \preceq_i^{(j)} T_{0 \rightarrow i}^{(j)}(\sigma_0^{(j)})$. Monotonicity of $T_{i \rightarrow i+1}^{(j)}$ gives $T_{i \rightarrow i+1}^{(j)}(\sigma_i^{(j)}) \preceq_{i+1}^{(j)} T_{0 \rightarrow i+1}^{(j)}(\sigma_0^{(j)})$, and (56) with transitivity gives the claim for $i + 1$.

Proposition 3 is order-theoretic: it requires monotonicity of the translations and proves a bound in the target

	PoC [10]	Relational PIC [11]	COSS
Event domain	$O \times R \times L$	execution occurrences indexed by causal histories	$\mathcal{B} \times L$
Carried state	$C \subseteq O \times R$	$(C, [\rho]_{\text{sem}})$	$\Sigma \in \prod_j X_j$
Order	$\subseteq$	$\subseteq$ and $\Rightarrow$, componentwise	$\preceq_j$, componentwise
Continuity result	origin-bounded authority	Dual Monotonicity and Combined Closure	Product Continuity
Separation result	lineage-invariant policies cannot individuate occurrences	quotient factorization and observation-based indistinguishability	Projection Factorization and lineage-forgetting blindness

Table 1: The PIC series as a layered continuity construction.

orders. A stronger claim that a translation preserves the intended security meaning across heterogeneous behavior universes requires a separate semantic relation between those universes. This paper does not define such a cross-domain semantic soundness condition; an overly permissive semantic translation is therefore a policy or deployment error rather than a failure of the order-theoretic continuity theorem.

13 Ordinary Continuation versus Authorized Relaxation

Not every legitimate security transition is monotone. Declassification may relax an IFC constraint; a new authority grant may enlarge an authority state; a policy administrator may broaden an admissible geographic region.

COSS therefore distinguishes two classes of transition.

Definition 9 (Ordinary continuation). *An ordinary continuation is a PoR-linked transition satisfying $\Sigma_{i+1} \preceq \Sigma_i$ (or the translated form of (56)).*

Definition 10 (Authorized relaxation event). *An authorized relaxation event is an explicitly distinguished causal event whose validity is justified by a separate authority or policy rule and which may replace one or more coordinates by states not below their predecessors.*

The point is not to prohibit declassification, re-authorization, or policy change. It is to prevent those events from being confused with ordinary continuation. A security state may become more permissive only at a boundary whose authority to perform that relaxation is itself explicit and auditable.

This mirrors the distinction already present in PIC between attenuation within one lineage and the creation of new authority at an origin or explicit future composition boundary.

Relaxation events are still causal events: they are PoR-linked to their predecessor, and only the monotonicity conjunct is replaced by the separate authorization rule.

Proposition 4 (Segment continuity). *Let a PoR-linked lineage contain authorized relaxation events at indices*

$1 \leq r_1 < \dots < r_k$, where the event at r_m is the transition from s_{r_m-1} to s_{r_m} , all other transitions being ordinary continuations. For every i , let $r(i)$ be the largest $r_m \leq i$, or 0 if none exists. Then

$$\Sigma_i \preceq \Sigma_{r(i)}. \quad (59)$$

The statement assumes a homogeneous state space along the segment. For segments containing translated hops, the corresponding bound is the one given by Proposition 3.

Proof. If $r(i) = i$, the claim is reflexivity. Otherwise, every transition leaving $s_{r(i)}$ and ending at or before s_i is an ordinary continuation, because $r(i)$ is the most recent relaxation-entry index. Product Continuity applied to the segment from $s_{r(i)}$ through s_i gives $\Sigma_i \preceq \Sigma_{r(i)}$.

Origin-boundedness is therefore replaced, in the presence of relaxation, by boundedness from the most recent explicitly authorized relaxation. A relaxation cannot be introduced by ordinary continuation; it can only restart the bound at an explicit, auditable point.

14 Composition and DAG-Shaped Causality

The present draft, like the existing PIC theory, states the core result first for a linear lineage. Multiple-cause effects require a DAG-shaped causal model and an explicit composition rule [11].

For a join with parents carrying states $\Sigma^{(1)}, \dots, \Sigma^{(k)}$, the critical requirement is that composition not erase the provenance of the coordinates being combined. A future composition operator

$$\kappa : X^k \rightarrow X_\kappa \quad (60)$$

must specify which jointly justified possibilities are admitted at the new execution node and must make any increase in the set of participating origins an explicit event rather than an ordinary continuation.

This is particularly important for product invocation. A runtime invocation may depend jointly on an authority

lineage and a product/provenance lineage. The invocation node should record both causes rather than retagging either parent state as if it originated in the join.

A complete DAG theory is left for future work; no general join operator is claimed here.

Miller and Shapiro use the phrase “mutually suspicious composition” for the coexistence of independently motivated access abstractions over one capability graph [6]. That notion is related in spirit but is not identified here with COSS’s future DAG composition: the latter concerns a causal join with multiple parent histories and requires an explicit rule for combining their carried security states. Keeping the two notions distinct avoids importing a join semantics that the present draft has not proved.

15 Related Work

Bell–LaPadula. Bell and LaPadula model secure system states together with rules of operation that preserve security under state transitions [2]. COSS does not replace those rules or claim that Bell–LaPadula is merely a set-inclusion policy. Rather, an effective Bell–LaPadula security state may serve as one coordinate when its local refinement relation and admitted-event semantics are specified; COSS adds the explicit causal-event axis and the product transport rule used when that state must coexist with other security states.

Denning’s lattice model. Denning organizes security classes as a lattice whose order is justified by the semantics of permitted information flow [3]. This is a direct example of an ordered security domain, but the IFC semantics remain Denning’s: COSS supplies neither the lattice nor a program analysis for discovering flows. It supplies a way to carry such an ordered state together with other coordinates along one causal execution.

Execution-history access control. Abadi and Fournet make run-time rights depend on execution history rather than only the current stack [7]. In their model, ordinary execution automatically intersects current rights with the static rights of code that runs, while an explicit special operation may modify current rights, at most restoring the static rights of that code and subject to further constraints. This is structurally close to the distinction here between ordinary non-expansive continuation and an explicit relaxation event. The models nevertheless have different domains: their construction computes a rights state for code execution history, whereas COSS abstracts over an arbitrary finite product of security-state coordinates and takes sound PoR as an explicit premise relating the modeled execution occurrences.

Decentralized information-flow control. Myers and Liskov define decentralized labels with owner-specific

reader sets, a restriction order that forms a security-class lattice, and explicit declassification that is legal only when the process has authority to act for the relevant owner [8]. Their restriction/declassification distinction is therefore an important prior instance of monotone security restriction plus explicitly authorized relaxation. COSS treats such an IFC state as one possible coordinate; it does not replace the label semantics, acts-for relation, or static flow analysis of that work.

Capability systems and access abstractions. Miller, Yee, and Shapiro show that conclusions about capability systems depend on the capability model and on distinctions such as data transfer versus capability transfer, designation with authority, and access-controlled delegation channels [5]. Miller and Shapiro further show that security-enforcing program behavior and access abstractions can restrict authority beyond what a protection-graph-only analysis exposes [6]. COSS’s claim is not that these systems lack context, flow control, or causal reasoning. Its contribution is a uniform event/state construction and a factorization result: whenever a target verdict depends on a distinction erased by an observation, a decision invariant under that observation cannot reproduce the target verdict.

16 Enforcement Assumptions

The mathematics constrains accepted transitions; it does not make causal attribution true by itself. The existing PIC theory already separates the continuity invariant from complete mediation, sound PoR verification, and causal attribution [11]. COSS inherits the same boundary.

For a deployment-level claim that an actual effect satisfies all carried security systems, at least the following are required:

1. every protected effect is mediated by an enforcement path that reads the relevant product coordinates;
2. PoR soundly binds each accepted transition to its actual causal predecessor;
3. each coordinate observation and translation is trustworthy enough for the theorem being claimed; and
4. any relaxation or composition event is separately authorized under the semantics of the coordinates it changes.

Information that influences later behavior through unobserved mutable state, queues, caches, timers, covert channels, or other unmodeled causes remains outside the guarantee unless the execution model or information-flow tracking brings those causes into the mediated causal structure.

17 Limitations and Open Questions

Several issues remain open. In particular, the present results leave four mathematical tasks as an explicit research agenda:

1. a DAG composition theorem with a join operator that preserves coordinate provenance and a stated non-expansion property;
2. a theory of causal attribution that states when an effect may be assigned to a history beyond explicit mediated handoffs;
3. semantic correctness conditions for translations between heterogeneous behavior and state spaces; and
4. an algebra of authorized relaxations, including their authority, composition, and interaction with later continuity bounds.

Choice of coordinates. The theory does not decide which security systems must be included. Omitting a coordinate is safe only for properties invariant under that omission; the projection theorem makes this requirement explicit.

Order soundness. The meaning of $\preceq_j$ is domain-specific. A badly chosen order can be mathematically monotone while failing to represent the intended security semantics.

Non-monotone security mechanisms. Declassification, endorsement, privilege grant, revocation semantics, and administrative policy changes need explicit transition classes rather than being hidden inside ordinary continuation.

DAG composition. Fan-out, fan-in, joins, and multi-cause effects require a partial-order causal model and a sound composition algebra.

Semantic soundness of heterogeneous translations. Translated Continuity proves preservation relative to the supplied ordered translation maps. Relating states and behaviors across different local vocabularies requires an additional cross-domain semantic relation and a soundness condition for each translation; those are left open here.

Authority for relaxation. The theory treats authorization of a relaxation as a separate premise. A deployment may represent that authority as another carried coordinate, but doing so does not eliminate the need for an ultimate trust or policy assumption about who may relax that coordinate. COSS does not claim to solve this regress internally.

Implementation evidence. The theory is agnostic about whether evidence is carried in tokens, attestations, trusted-runtime state, gateways, proof-carrying objects, or other mechanisms. Representation is secondary to whether the enforcement decision observes the coordinates required by the property being enforced.

18 Discussion

Scope of the framework. Across the three-paper sequence, the common primitives are an execution occurrence, a causal history, PoR between adjacent occurrences, an ordered state carried by the history, ordinary continuation that does not expand that state, and an explicit transition class for authorized relaxation. The continuity and separation results arise from these primitives plus the stated semantic and enforcement assumptions. The order theory used here is elementary; the contribution claimed is the choice of authorization domain and the causal transport discipline, not a new result in order theory.

The proposed shift is small syntactically but large semantically. Instead of asking one security model to become universal, COSS treats each model as a coordinate over the same causal execution.

The general invariant is:

Ordinary causal transport may not make any carried security state more permissive according to that security system’s own refinement order. Any relaxation must be an explicit, separately authorized causal event.

This separates three concerns that are often conflated:

1. *what* a security system considers more or less permissive;
2. *which* security systems a decision must observe; and
3. *whether* the observed state belongs to the causal execution that is actually producing the effect.

Capability authority, Bell–LaPadula-style information-flow constraints, relational executor admissibility, provenance, geography, and other controls can therefore remain distinctly meaningful. They need not be merged into a single ontology. Their product is carried by the execution, while PoR provides the causal axis that keeps the evolving states attached to one history.

An access abstraction need not always become a new coordinate. It may instead be the concrete mechanism that realizes a coordinate’s semantic extension, or it may restrict the admissible subset Y of product states. This is important for reading the capability literature correctly: the factory, Caretaker, and data-diode constructions of Miller and Shapiro constrain authority by program behavior, even when a coarser projection of the protection state would admit more possibilities [6]. COSS abstracts over

the implementation choice and asks only which distinctions the receiving decision actually observes and preserves along the causal execution.

19 Conclusion

PIC placed authority-use events on the causal axis, in $O \times R \times L$. COSS keeps that axis and replaces the privilege behavior space $O \times R$ with the product behavior space $\mathcal{B}$, yielding events in $\mathcal{B} \times L$, while each lineage endpoint carries a product security state in $X = \prod_j X_j$. Ordinary continuation requires componentwise refinement of that carried state along verified causal edges. The resulting Product Continuity theorem is immediate from transitivity, but its consequence is broad: security systems with different semantics can be preserved together without requiring one to subsume the others.

The projection theorem explains the corresponding limitation. A decision which observes only a subset of coordinates factors through that projection and cannot enforce a property that depends on information erased by it. This provides a common mathematical lens for earlier results that otherwise appear to pit capability authority against information-flow control.

PIC emerges as the authority specialization of this construction, and its relational extension is already a nontrivial product instance. Together, the three papers develop a layered causal theory of authorization and security-state continuity, of which COSS is the general product form developed here. The broader research question is therefore no longer only how authority propagates without expansion, but how heterogeneous security states can remain causally bound and jointly non-expansive across distributed execution.

References

- [1] D. E. Bell and L. J. LaPadula, “Secure Computer Systems: Mathematical Foundations,” The MITRE Corporation, ESD-TR-73-278, Vol. I, 1973.
- [2] L. J. LaPadula and D. E. Bell, “Secure Computer Systems: A Mathematical Model,” The MITRE Corporation, ESD-TR-73-278, Vol. II, 1973.
- [3] D. E. Denning, “A Lattice Model of Secure Information Flow,” *Communications of the ACM*, vol. 19, no. 5, pp. 236–243, 1976, doi: 10.1145/360051.360056.
- [4] W. E. Boebert, “On the Inability of an Unmodified Capability Machine to Enforce the *-Property,” in *Proceedings of the 7th DoD/NBS Computer Security Conference*, Gaithersburg, MD, USA, Sep. 24–26, 1984, pp. 291–293. [Online]. Available: <https://csrc.nist.gov/files/pubs/conference/1984/09/24/7th-dodnbs-computer-security-conference-final/docs/1984-7th-conference-proceedings.pdf>.
- [5] M. S. Miller, K.-P. Yee, and J. S. Shapiro, “Capability Myths Demolished,” Technical Report SRL2003-02, Systems Research Laboratory, Department of Computer Science, Johns Hopkins University, Mar. 2003.
- [6] M. S. Miller and J. S. Shapiro, “Paradigm Regained: Abstraction Mechanisms for Access Control,” in *Advances in Computing Science – ASIAN 2003: Programming Languages and Distributed Computation*, Lecture Notes in Computer Science, vol. 2896, Springer, 2003, pp. 224–242, doi: 10.1007/978-3-540-40965-6_15.
- [7] M. Abadi and C. Fournet, “Access Control Based on Execution History,” in *Proceedings of the 10th Annual Network and Distributed System Security Symposium (NDSS)*, 2003.
- [8] A. C. Myers and B. Liskov, “A Decentralized Model for Information Flow Control,” in *Proceedings of the 16th ACM Symposium on Operating Systems Principles (SOSP)*, pp. 129–142, 1997, doi: 10.1145/268998.266669.
- [9] N. Hardy, “The Confused Deputy (or Why Capabilities Might Have Been Invented),” *ACM SIGOPS Operating Systems Review*, vol. 22, no. 4, pp. 36–38, 1988.
- [10] N. Gallo, “Proof-of-Continuity: A Temporal Model for Authority Propagation in Distributed Systems and AI Agents,” arXiv:2607.08906, 2026.
- [11] N. Gallo, “Beyond Proof of Possession: A Relational Theory of Authority,” manuscript, 2026.